

FULL STACK DEVELOPMENT

FASE 4 | AULA 04 -

CONEXÃO COM APIS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	26
REFERÊNCIAS.....	27

EMSE

O QUE VEM POR AÍ?

Nesta aula, você terá um contexto de como funcionam as conexões com APIs em aplicativos mobile. Abordaremos boas práticas, alguns dos métodos de navegação, configurações necessárias para efetuarmos o uso de um dos métodos e como tratar alguns casos de erro.



HANDS ON

Nesta aula, abordaremos como efetuar a comunicação com APIs externas em nossa aplicação, a configuração que devemos aplicar para utilizarmos a API e como fazer tratativa de alguns erros.



SAIBA MAIS

Segue abaixo o código trabalhado no Hands on:

<code>npm install @react-navigation/native-stack</code>
<code>npm install @react-native-picker/picker</code>
<code>npx expo install axios</code>
<pre>import { Text, View, TextInput, Button, Alert, StyleSheet, TouchableOpacity } from "react-native"; import { StatusBar } from "expo-status-bar"; import { useState } from "react"; import { LinearGradient } from 'expo-linear-gradient' import Header from '.././././components/shared/Header' import { useNavigation } from '@react-navigation/native' export default function Login() { const [user , setUser] = useState("") const [password , setPassword] = useState("") const navigation = useNavigation() const handleInputChange = (inputText) =>{ setText(inputText) } const handleButtonPress = () => { Alert.alert('Credenciais Digitadas', `Usuário: \${user}\nSenha: \${password}`,</pre>

```
[
  {
    text: 'OK',
    onPress: () => console.log('OK Pressed'),
    style: 'cancel',
    accessibilityLabel: 'Fechar Alerta',
  },
],
{ cancelable: true }
);

navigation.navigate('screens/Home/index',{userName: user})
}

return (
  <LinearGradient
    style={styles.container}
    colors={['#87CEEB','#FFFFFF']}

  >

  <Header/>

  <View style={styles.content}>

  <Text>Clima Tempo</Text>

  <Text>Digite seu usuário</Text>

  <TextInput style={styles.input}

    onChangeText={({inputText})=>{setUser(inputText)}}
  />
</View>
</LinearGradient>
);
```

```
        value={user}
      />
      <Text>Digite sua senha</Text>
      <TextInput/>
      <TextInput style={styles.input}
        onChangeText={({inputText})=>{setPassword(inputText)}}
        value={password}
      />
      <StatusBar style="auto" />
      <TouchableOpacity style={styles.button} onPress={handleButtonPress}>
        <Text>Enviar</Text>
      </TouchableOpacity>
    </View>
  </LinearGradient>
);
}

const styles = StyleSheet.create({
  container:{
    flex:1,
    backgroundColor: '#fff',
  },
  content:{
    flex:1,
    alignItems: 'center',
```

```
      justifyContent: 'center',  
    },  
    input: {  
      height: 40,  
      borderColor: 'gray',  
      borderWidth: 1,  
      marginBottom: 10,  
      paddingHorizontal: 10,  
      width: '100%',  
      borderRadius: 25,  
    },  
    button: {  
      backgroundColor: 'inherit',  
      borderColor: 'grey',  
      borderWidth: 1,  
      width: '50%',  
      alignItems: 'center',  
      padding: 5,  
    }  
  })
```

```
import {View, Text, StyleSheet} from "react-native";  
  
import { WeatherCardInfo } from '../components/WeatherCardInfo'  
  
import { TemperatureCard } from '../components/TemperatureCard'  
  
import { useEffect, useState } from "react";  
  
import getWeather from "@api/weatherapi";
```



```
const weather = {  
  "coord": {  
    "lon": 10.99,  
    "lat": 44.34  
  },  
  "weather": [  
    {  
      "id": 501,  
      "main": "Rain",  
      "description": "moderate rain",  
      "icon": "10d"  
    }  
  ],  
  "base": "stations",  
  "main": {  
    "temp": 298.48,  
    "feels_like": 298.74,  
    "temp_min": 297.56,  
    "temp_max": 300.05,  
    "pressure": 1015,  
    "humidity": 64,  
    "sea_level": 1015,  
    "grnd_level": 933  
  },  
  "visibility": 10000,  
  "wind": {
```

```
"speed": 0.62,  
  "deg": 349,  
  "gust": 1.18  
},  
  "rain": {  
    "1h": 3.16  
  },  
  "clouds": {  
    "all": 100  
  },  
  "dt": 1661870592,  
  "sys": {  
    "type": 2,  
    "id": 2075663,  
    "country": "IT",  
    "sunrise": 1661834187,  
    "sunset": 1661882248  
  },  
  "timezone": 7200,  
  "id": 3163858,  
  "name": "Zocca",  
  "cod": 200  
};
```

```
export default function Home({route}){  
  const [currentTemperature, setCurrentTemperature] = useState()
```

```
const [temperatureMin, setTemperatureMin] = useState()
const [temperatureMax, setTemperatureMax] = useState()
const [feelsLike, setFeelsLike] = useState()
const [selectedState, setSelectedState] = useState('São Paulo');
const [wind, setWind] = useState()
const [pressure, setPressure] = useState()
const [humidity, setHumidity] = useState()

const handleStateChange = (newState) => {
  setSelectedState(newState);
};

useEffect(() =>{
  fetchData()
},[])

useEffect(() =>{
  fetchData()
},[selectedState])

function convertKelvinToC(kelvin){
  return "+parseInt(kelvin - 273)
}

const { userName } = route.params

async function fetchData(){
```

```
try{
  const currentWeather = await getWeather(selectedState);
  if(currentWeather !== null){
    setTemperatureMin(convertKelvinToC(currentWeather[0]));
    setTemperatureMax(convertKelvinToC(currentWeather[1]));
    setWind(currentWeather[2])
    setHumidity(currentWeather[3])
    setCurrentTemperature(convertKelvinToC(currentWeather[4]));
    setFeelsLike(convertKelvinToC(currentWeather[5]));
    setPressure(currentWeather[6])
  }
}catch(error){
  console.log('Error fetching weather data: ', error)
}
}
return(
  <View style={styles.homeContainer}>
    <WeatherCardInfo user={userName} temperature={` ${currentTemperature}°` }
    selectedState={selectedState} onStateChange={handleStateChange}/>
    <View style={styles.otherTemperatures}>
      <TemperatureCard temperature={` ${temperatureMin}°`}/>
      <TemperatureCard temperature={` ${temperatureMax}°`}/>
    </View>
    <View style={styles.infoField}>
      <Text style={styles.infoText}>Mais Detalhes</Text>
      <View style={[styles.detailsContent, { borderTopStartRadius: 8,
```

```
borderTopEndRadius: 8 ]}]>

    <View style={styles.details}>

        <Text style={styles.descriptionTitle}>Vel. Vento</Text>

        <Text>{wind}</Text>

    </View>

    <View style={styles.details}>

        <Text style={styles.descriptionTitle}>Umidade</Text>

        <Text>{humidity}%</Text>

    </View>

</View>

<View style={[styles.detailsContent, { borderBottomStartRadius: 8,
borderBottomEndRadius: 8 }]}>

    <View style={styles.details}>

        <Text style={styles.descriptionTitle}>Sensação Térmica</Text>

        <Text>{feelsLike}%</Text>

    </View>

    <View style={styles.details}>

        <Text style={styles.descriptionTitle}>Pressão Atmosferica</Text>

        <Text>{pressure}%</Text>

    </View>

</View>

</View>

)
}
```

```
const styles = StyleSheet.create({
  homeContainer:{
    flex:1
  },
  otherTemperatures:{
    display:'flex',
    flexDirection:'row',
    justifyContent:'space-around',
    alignItems:'center',
    marginTop: 20
  },
  infoField:{
    display:'flex',
    justifyContent:'center',
    alignItems:'center',
    width:'100%',
    marginTop:10
  },
  infoText:{
    fontSize:14,
    fontWeight:'bold',
  },
  detailsContent:{
    display:'flex',
    width:'80%',
    flexDirection: 'row',
```

```
    justifyContent:'space-between',  
    backgroundColor:'#87CEEB',  
  },  
  details:{  
    width:'50%',  
    padding: 5  
  },  
  descriptionTitle:{  
    fontSize:14,  
    fontWeight:'bold',  
  },  
})
```

```
import { Feather } from "@expo/vector-icons";  
import { LinearGradient } from "expo-linear-gradient";  
import { View, Text, StyleSheet } from "react-native";  
import { Picker } from '@react-native-picker/picker'  
  
export function WeatherCardInfo({user, temperature, selectedState,  
onStateChange}){  
  const states = [  
    { uf: 'AC', name: 'Acre' },  
    { uf: 'AL', name: 'Alagoas' },  
    { uf: 'AP', name: 'Amapa' },  
    { uf: 'AM', name: 'Amazonas' },  
    { uf: 'BA', name: 'Bahia' },
```

```
{ uf: 'CE', name: 'Ceara' },  
{ uf: 'DF', name: 'Distrito Federal' },  
{ uf: 'ES', name: 'Espírito Santo' },  
{ uf: 'GO', name: 'Goias' },  
{ uf: 'MA', name: 'Maranhao' },  
{ uf: 'MT', name: 'Mato Grosso' },  
{ uf: 'MS', name: 'Mato Grosso do Sul' },  
{ uf: 'MG', name: 'Minas Gerais' },  
{ uf: 'PA', name: 'Para' },  
{ uf: 'PB', name: 'Paraiba' },  
{ uf: 'PR', name: 'Parana' },  
{ uf: 'PE', name: 'Pernambuco' },  
{ uf: 'PI', name: 'Piaui' },  
{ uf: 'RJ', name: 'Rio de Janeiro' },  
{ uf: 'RN', name: 'Rio Grande do Norte' },  
{ uf: 'RS', name: 'Rio Grande do Sul' },  
{ uf: 'RO', name: 'Rondônia' },  
{ uf: 'RR', name: 'Roraima' },  
{ uf: 'SC', name: 'Santa Catarina' },  
{ uf: 'SP', name: 'Sao Paulo' },  
{ uf: 'SE', name: 'Sergipe' },  
{ uf: 'TO', name: 'Tocantins' },  
];
```

```
const actualDate = new Date()
```

```
const dateFormatted = actualDate.toLocaleDateString('pt-BR',{
```



```
    day: '2-digit',
    month: '2-digit',
    year: 'numeric',
  })
  return(
    <LinearGradient style={styles.container}
      colors={['#87CEEB', '#ff23']}
      start={{ x: 0.5, y: 0 }}
      end={{ x: 0.5, y: 1 }}
    >
    <View style={styles.header}>
      <Text style={styles.paragraph}>Boas Vindas {user}</Text>
      <Text style={styles.paragraph}>{dateFormatted}</Text>
    </View>
    <View>
      <Text>Selecione um Estado:</Text>
      <Picker
        selectedValue={selectedState}
        onValueChange={(itemValue, itemIndex) =>
          onStateChange(itemValue)}
        itemStyle={{backgroundColor:'inherit'}}
      >
        {
          states.map((state, index)=>(
            <Picker.Item
              key={index}
```

```
        label={state.name}
        value={state.name}
        color="#000"
      />
    ))
  }
</Picker>
</View>
<View style={styles.weather}>
  <Feather name="sun" size={50} color="#fff"/>
  <Text style={styles.weatherText}>{temperature}</Text>
</View>
</LinearGradient>
)
}
const styles = StyleSheet.create({
  container: {
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
    width: '100%',
  },
  header: {
    display: 'flex',
    flexDirection: 'row',
    justifyContent: 'space-between',
```

```
width: '100%'

},

weather:{

  marginTop:20,

  display:'flex',

  justifyContent:'center',

  alignItems:'center'

},

weatherText:{

  fontSize: 20,

  fontWeight: 'bold',

  color:'#fff'

},

paragraph: {

  fontSize: 18,

  textAlign: 'center',

  color:'#fff'

},

picker: {

  width: '100%',

  height: 50,

  borderColor: '#ccc',

  borderWidth: 1,

  borderRadius: 5,

  backgroundColor:'inherit'

},
```

```
pickerText:{
  color:'#fff'
},
item: {
  fontSize: 16,
  color:'#fff',
  padding:0,
},
selectedText: {
  marginTop: 20,
  fontSize: 18,
},
});
```

```
import { Feather } from "@expo/vector-icons";
import { LinearGradient } from "expo-linear-gradient";
import { View, Text, StyleSheet } from "react-native";

export function TemperatureCard({temperature}){
  return(
    <LinearGradient style={styles.container}
      colors={['#87CEEB', '#ff23']}
      start={{ x: 0.5, y: 0 }}
      end={{ x: 0.5, y: 1 }}
    >
```

```
<View style={styles.weather}>
  <Feather name="sun" size={50} color="#fff"/>
  <Text style={styles.weatherText}>{temperature}</Text>
</View>
</LinearGradient>
)
}
const styles = StyleSheet.create({
  container: {
    alignItems: 'center',
    justifyContent: 'center',
    padding: 20,
    width: '45%',
  },
  weather:{
    marginTop:20,
    display:'flex',
    justifyContent:'center',
    alignItems:'center'
  },
  weatherText:{
    fontSize: 20,
    fontWeight: 'bold',
    color:'#fff'
  },
  paragraph: {
```

```
fontSize: 18,  
textAlign: 'center',  
color: '#fff'  
},  
});
```

Conexão com APIs

A conexão com APIs é uma parte essencial do desenvolvimento de aplicativos modernos, e isso se aplica igualmente ao desenvolvimento com React Native.

Como já mencionado, React Native é uma biblioteca JavaScript para criar aplicativos móveis nativos para Android e iOS, e sua flexibilidade e eficiência tornaram-no uma escolha popular para pessoas desenvolvedoras em todo o mundo. Vamos explorar, nesta seção, os principais aspectos da conexão com APIs em aplicativos React Native.

Fundamentos da conexão com API em React Native

A integração de uma API em um aplicativo React Native envolve vários passos essenciais. Primeiramente, é necessário configurar o ambiente de desenvolvimento, o que inclui a instalação do Node.js, npm/yarn e do React Native CLI. Essas ferramentas são fundamentais para criar e gerenciar projetos React Native.

Realização de requisições HTTP

Para se comunicar com uma API, o aplicativo React Native precisa realizar requisições HTTP. Isso pode ser feito usando o método `fetch`, que é nativo do JavaScript, ou utilizando bibliotecas como o `axios`, que simplificam o processo de requisição e tratamento de respostas.

As requisições HTTP podem ser de diferentes tipos, como GET, POST, PUT e DELETE, dependendo da ação que o aplicativo deseja realizar na API. Por exemplo, uma requisição GET é usada para obter dados da API, enquanto uma

requisição POST é usada para enviar dados para a API, como ao criar um novo registro.

Tratamento de respostas da API

O tratamento de respostas da API é crucial para garantir o funcionamento correto do aplicativo. As respostas da API geralmente são retornadas no formato JSON, e o aplicativo React Native deve ser capaz de interpretar esses dados e agir de acordo.

Além disso, é importante lidar com códigos de status HTTP retornados pela API, como 200 (OK), 404 (Não encontrado), 500 (Erro interno do servidor), entre outros. O tratamento adequado desses códigos permite ao aplicativo reagir de maneira apropriada, exibindo mensagens de erro, redirecionando o usuário, ou realizando outras ações necessárias.

Autenticação e autorização

Muitas APIs requerem autenticação para acessar determinados recursos. Isso pode ser feito por meio de tokens JWT (JSON Web Tokens), OAuth ou outros métodos de autenticação. O aplicativo React Native precisa implementar esses mecanismos de autenticação de forma segura, garantindo que apenas usuários autorizados tenham acesso aos recursos protegidos da API.

O armazenamento seguro de tokens de autenticação é outra consideração importante. O uso de ferramentas como AsyncStorage ou SecureStore no React Native permitem armazenar esses tokens de forma segura no dispositivo do usuário.

Gerenciamento de estado global

À medida que um aplicativo cresce em complexidade, o gerenciamento de estado torna-se crucial. No contexto de conexão com API, é comum ter estados relacionados a carregamento de dados, sucesso ou falha de requisições, dados retornados pela API, entre outros.

O React Native oferece a Context API para gerenciamento de estado global de forma mais simples, permitindo que diferentes partes do aplicativo acessem e modifiquem o estado compartilhado. Além disso, bibliotecas externas como Redux ou MobX são amplamente utilizadas para gerenciamento avançado de estado em aplicativos React Native.

Melhores práticas e segurança

Ao integrar uma API em um aplicativo React Native, é importante seguir melhores práticas de desenvolvimento. Isso inclui separar responsabilidades em componentes, serviços de API e gerenciar o fluxo de dados de maneira organizada e eficiente.

A segurança também é uma consideração crítica. O uso de HTTPS para comunicação segura com a API, além de práticas como validação de entrada, sanitização de dados e tratamento seguro de informações sensíveis são aspectos que não podem ser negligenciados.

Testes, depuração e documentação

Por fim, é essencial realizar testes unitários e de integração para garantir que a integração com a API funcione conforme o esperado em diferentes cenários. Ferramentas de depuração como Reactotron e as ferramentas de desenvolvimento do navegador são úteis para identificar e corrigir problemas durante o desenvolvimento.

A documentação da API também desempenha um papel crucial. Utilizar ferramentas como Swagger ou OpenAPI para documentar a API facilita a integração e compreensão dos endpoints disponíveis, parâmetros necessários e formatos de dados esperados ou retornados.

Formas de conexão com APIs

Uso do Axios em React Native

O Axios é uma escolha popular para realizar conexões com APIs em aplicativos React Native devido à sua facilidade de uso e recursos avançados. Ao utilizar o Axios, quem desenvolve pode aproveitar benefícios como:

- **Sintaxe simples:** o Axios oferece uma sintaxe simples e intuitiva para fazer requisições HTTP, o que facilita a implementação e compreensão do código.
- **Tratamento de respostas:** além de realizar requisições, o Axios simplifica o tratamento de respostas, permitindo o acesso aos dados da resposta de forma direta e organizada.

- **Interceptadores:** os interceptadores do Axios são úteis para adicionar cabeçalhos personalizados, tratar erros de forma global e realizar transformações nos dados antes do envio ou após o recebimento da resposta.
- **Cancelamento de requisições:** o Axios possui suporte nativo para o cancelamento de requisições, o que é útil em situações onde requisições precisam ser interrompidas, como ao navegar para outra tela ou ao finalizar uma operação.

Outras formas de conexão com APIs em React Native

Além do Axios, existem outras formas de conectar aplicativos React Native a APIs, cada um com suas vantagens e cenários de uso:

- **Fetch API:** a Fetch API é uma alternativa nativa do navegador que também está disponível em ambientes React Native. Ela oferece uma abordagem moderna e simplificada para fazer requisições HTTP, com suporte a promises para tratamento assíncrono de respostas.
- **XMLHttpRequest (XHR):** embora seja menos comum em projetos React Native, o XMLHttpRequest é uma API antiga do JavaScript que também pode ser utilizada para fazer requisições HTTP assíncronas. No entanto, o Axios e a Fetch API são geralmente preferidos devido à sua interface mais moderna e fácil de usar.
- **Bibliotecas específicas:** dependendo das necessidades do projeto, é possível encontrar bibliotecas específicas para tipos de APIs ou integrações. Por exemplo, bibliotecas para trabalhar com GraphQL, WebSocket, APIs de serviços como Firebase, AWS, entre outras, oferecem recursos específicos e simplificados para esses cenários.

A escolha entre o Axios e outras formas de conexão com APIs em React Native depende das preferências da equipe de desenvolvimento, dos recursos necessários para o projeto e das funcionalidades oferecidas por cada opção. O importante é utilizar a abordagem mais adequada para garantir uma integração eficiente e segura com a API utilizada pelo aplicativo.

O QUE VOCÊ VIU NESTA AULA?

Na aula, aprendemos sobre o funcionamento das conexões com APIs em aplicativos mobile, destacando boas práticas de segurança, métodos de navegação e configurações necessárias para integração.

Foi destacada a importância da autenticação, navegação intuitiva e como lidar com casos de erro durante as conexões para garantir uma experiência consistente aos usuários, mesmo em situações adversas.

Por fim, saímos com uma compreensão ampla sobre como desenvolver aplicativos mobile robustos e eficientes, conectados de forma segura com serviços externos por meio de APIs.

REFERÊNCIAS

AXIOS. Documentação oficial. Disponível em: <https://axios-http.com/ptbr/docs/intro>.

Acesso em: 19 jul. 2024.

EXPO. Documentação oficial. Disponível em: <https://docs.expo.dev>. Acesso em: 19

jul. 2024.

REACT NATIVE. Documentação Oficial. Disponível em: <https://reactnative.dev/>.

Acesso em: 19 jul. 2024.

EXEMPLO

PALAVRAS-CHAVE

Palavras-chave: React Native. Axios. Fetch Api.

EMSE

POS TECH